



Desafio Node | E-commerce API | Loomi



Acordos

Durante essa etapa, queremos observar como você estrutura soluções, organiza prioridades e resolve problemas de forma autônoma. É a sua chance de mostrar como você pensa e constrói!

Organização e gerenciamento

Você tem total liberdade para organizar seu tempo e estrutura de trabalho. Queremos entender como você gerencia suas próprias atividades, define prioridades e se comunica de forma clara, de forma assíncrona através de documentações.

Você pode entrar em contato com a pessoa do time de processo seletivo caso surjam dúvidas técnicas que realmente sejam essenciais. A troca deve ser estratégica e pontual.

Entregáveis

- **Relatório de Progresso:**

- Link da plataforma de gestão de atividades utilizada para criar e acompanhar backlog
- Como organizou suas demandas e atividades.
- Como priorizou as entregas.

- Quais foram as principais dificuldades enfrentadas e como lidou com elas.
- O que faria diferente com mais tempo ou em um contexto real de projeto.
- **Entregáveis Técnicos:**
 - Garantir que foi solicitado no escopo do desafio.
- **#Essencial: Fluxo de Git**

Utilize Git para versionamento de código. Faça commits descritivos; crie uma branch para cada feature e abra um Pull Request para a branch principal ao finalizar (`develop` ou equivalente).

Isso é importante para organização de fluxo de trabalho e entendimento do histórico.



Prazo e Entrega

- Você tem até **25/01/2026 às 23:59** para realizar a entrega, 4 dias corridos a partir do recebimento do desafio
- **Enviar a pasta com o projeto completo como anexo, em formato zip, via e-mail para processoseletivo@loomi.com.br**



Processo Seletivo

Briefing da Solução

Construir uma API REST para um sistema de gerenciamento de um e-commerce simplificado. No sistema são gerenciados clientes, produtos e pedidos.

Esta aplicação será a base de uma aplicação de e-commerce, responsável por gerenciar todas as interações entre o usuário e a gestão de produtos.

A aplicação deve armazenar e devolver os dados do banco de dados quando solicitados.

▼ Requisitos por Prioridade

Obrigatórios

- Sistema de autenticação (login/registro)

- CRUD completo de Produtos
- CRUD completo de Clientes
- CRUD de Pedidos
- Validação de dados
- Tratamento de erros
- Documentação da API (Swagger)
- Testes unitários essenciais
- Configuração Docker

Importante → Core da Aplicação

- Sistema de carrinho de compras
- Controle de estoque
- Filtros nas listagens
- Sistema de permissões (admin/cliente)

Extras

- Relatórios de vendas
- Integração com email
- Deploy na AWS
- Clean Architecture

▼ Modelagem das Entidades

Usuário

- **ID:** Chave primária, única
- **Nome:** String
- **Email:** String, único
- **Senha:** String, armazenada como hash
- **Data de Criação:** Timestamp
- **Data de Atualização:** Timestamp
- **Tipo:** Enum (Admin/Cliente)

Cliente

- **ID do Cliente:** Chave primária, única
- **ID do Usuário:** Chave estrangeira referenciando Usuário
- **Nome Completo:** String
- **Contato:** String
- **Endereço:** Texto
- **Status:** Boolean (Ativo/Inativo)
- **Data de Criação:** Timestamp
- **Data de Atualização:** Timestamp

Produto

- **ID do Produto:** Chave primária, única
- **Nome do Produto:** String
- **Descrição:** Texto
- **Preço:** Decimal
- **Quantidade em Estoque:** Inteiro
- **Data de Criação:** Timestamp
- **Data de Atualização:** Timestamp

Pedido

- **ID do Pedido:** Chave primária, única
- **ID do Cliente:** Chave estrangeira referenciando Cliente
- **Status do Pedido:** Enum (Recebido, Em preparação, Despachado, Entregue)
- **Data do Pedido:** Timestamp
- **Total do Pedido:** Decimal

Item do Pedido

- **ID do Item:** Chave primária, única
- **ID do Pedido:** Chave estrangeira referenciando Pedido

- **ID do Produto:** Chave estrangeira referenciando Produto
- **Quantidade:** Inteiro
- **Preço por Unidade:** Decimal
- **Subtotal:** Decimal (calculado como Quantidade * Preço por Unidade)

Relacionamentos

- **Usuário-Cliente:** Um para um, um usuário pode ser um cliente
- **Cliente-Pedido:** Um para muitos, um cliente pode ter vários pedidos
- **Pedido-Item do Pedido:** Um para muitos, um pedido pode conter vários itens
- **Produto-Item do Pedido:** Um para muitos, um produto pode estar em vários itens de pedido

▼ Funcionalidades Detalhadas

Obrigatórias

Sistema de Autenticação

- Login com email e senha
- Registro de usuários clientes
- JWT para autenticação das rotas protegidas
- Hash de senha com bcrypt

Gestão de Clientes

- CRUD completo (Criar, Ler, Atualizar, Deletar)
- Cliente só pode editar seus próprios dados
- Admin pode gerenciar todos os clientes

Gestão de Produtos

- CRUD completo para produtos
- Admin pode gerenciar produtos
- Cliente pode apenas visualizar produtos
- Controle básico de estoque

Gestão de Pedidos

- Cliente pode criar pedidos
- Admin pode atualizar status dos pedidos
- Cálculo automático do total do pedido
- Redução do estoque ao confirmar pedido

Validação e Segurança

- Validação de dados de entrada (Yup ou similar)
- Tratamento de erros customizado
- Middleware de autenticação
- Sanitização de dados

Documentação e Testes

- Documentação da API com Swagger
- Testes unitários para funcionalidades críticas
- README com instruções de configuração e uso

Importantes

Sistema de Carrinho

- Adicionar/remover produtos do carrinho
- Atualizar quantidades
- Calcular total do carrinho

Filtros e Busca

- Filtros por preço, nome, disponibilidade
- Busca por cliente, data de pedido
- Paginação nas listagens

Sistema de Permissões

- Rotas protegidas por tipo de usuário
- Middleware de autorização

Extras

Relatórios

- Relatório de vendas por período
- Geração de arquivo CSV
- Endpoint para download de relatórios

Integrações

- Envio de email de confirmação
- Integração com serviço de pagamento simulado

Deploy e Infraestrutura

- Deploy na AWS EC2
 - Configuração de ambiente de produção
-

Uso de ferramentas de IA

O uso consciente e estratégico de ferramentas de IA é incentivado para potencializar o desenvolvimento, sem substituir a autoria e revisão humanas.

Ferramentas Recomendadas

- **Cursor:** Editor de código com integração de IA, permitindo edição por linguagem natural, sugestões contextuais e refatoração inteligente
- **ChatGPT (OpenAI):** Assistente versátil para geração de código, esclarecimento de dúvidas e brainstorming técnico
- **Claude (Anthropic):** Focado em segurança e alinhamento, útil para revisões e explicações detalhadas
- **Gemini (Google):** Modelo multimodal com integração ao ecossistema Google

Diretrizes de Uso

- **Objetividade nos Prompts:** Formule solicitações claras e específicas para obter respostas relevantes das ferramentas de IA
- **Revisão Humana:** Todo código gerado por IA deve ser cuidadosamente revisado e ajustado conforme necessário
- **Documentação:** No README do projeto, inclua uma seção detalhando:
 - Quais ferramentas de IA foram utilizadas

- Exemplos de prompts e resultados obtidos
 - Decisões tomadas com base nas sugestões da IA
 - **Ética e Autenticidade:** Utilize a IA como suporte, mantendo a autoria e responsabilidade pelo código desenvolvido
-

Pontos Avaliados

Serão observados os seguintes aspectos sobre o desenvolvimento do desafio:



Aspectos Técnicos

- Qualidade e organização do código
- Boas práticas de desenvolvimento (SOLID, Clean Code, etc.)
- Tratamento de erros e validação de dados
- Documentação da API
- Implementação de testes
- Uso de Git e Docker



Funcionalidades

- Implementação das funcionalidades obrigatórias
- Qualidade das funcionalidades importantes
- Implementação de funcionalidades extras (diferencial)



Gestão e Comunicação

- Organização e planejamento das atividades
- Comunicação durante o desenvolvimento
- Cumprimento de prazos e entregas



Uso de Ferramentas de IA

- Documentação do uso de IA no projeto
 - Qualidade das implementações desenvolvidas com auxílio de IA
-

Diferenciais (Extras)

- Deployar a API na AWS (ssh, EC2, security group)
 - Usar AWS S3 para armazenar arquivos estáticos
 - Utilizar CleanArch
 - Implementar cache (Redis)
 - Implementar rate limiting
 - Adicionar health checks
 - Implementar CI/CD
-

Definições gerais

- A aplicação deverá seguir os conceitos REST (verbos HTTP, status de resposta HTTP, etc)
 - Todos os endpoints devem estar documentados utilizando Swagger
 - Devem ser implementados testes unitários para as funcionalidades mais críticas
 - Use Docker para containerização da aplicação
-

Bom desenvolvimento! 🚀